



Environment Strategy and Production Rollout Plan

Prepared by – **iorta TechNXT**

Version - **1.0**

Date - **04 October 2026**

TABLE OF CONTENTS

1. Document Control.....	4
2. Acronyms.....	4
3. Introduction.....	5
3.1 Purpose.....	5
3.2 Sources and labels.....	5
3.3 Related documents.....	6
4. Environments.....	7
4.1 The five environment roles.....	7
4.2 Environment sets by broker size.....	7
4.3 Pre-Prod lifecycle.....	8
4.4 Hosting by option.....	8
4.5 Sizing and hosting cost.....	9
4.6 Access control.....	10
4.7 Refresh policy.....	10
4.8 Pre-Prod: what must be identical to Production.....	11
4.9 Data masking.....	12
5. Moving code and database scripts.....	13
5.1 Git branching and release tagging.....	13
5.2 What CI checks.....	13
5.3 Release pipeline and approval gates.....	13
5.4 Build once, promote the same artefact.....	13
5.5 Front-end build and runtime configuration.....	14
5.6 Schema migrations.....	14
5.7 Reference seeds.....	15
5.8 Environment variables and secrets.....	15
5.9 Promotion steps.....	16
5.10 Change and release management.....	17
5.11 Rollback.....	17
6. Moving configuration.....	18
6.1 The configuration workbook.....	18
6.2 Promotion procedure.....	18
6.3 Not in the workbook.....	19
6.4 Configuration freeze.....	20
6.5 Comparing environments.....	20
6.6 Comparison report.....	20
7. Client data flow.....	21

7.1 The two workbooks the broker fills.....	21
7.2 Mock loads and the final load.....	21
7.3 Keeping migrated and new business apart and in step.....	22
8. Smoke test and go-live.....	24
8.1 Order of events.....	24
8.2 Hypercare.....	24
9. Cutover runbook.....	26
9.1 Conventions.....	26
9.2 T-30 to T-1.....	26
9.3 T to T+30.....	28
10. RACI and checklists.....	30
10.1 RACI for environment and release activities.....	30
10.2 Checklist: system integration test (Dev or SIT).....	30
10.3 Checklist: Dev or SIT to UAT.....	31
10.4 Checklist: UAT to Production and the Pre-Prod rehearsal.....	31
10.5 Checklist: go-live in Production.....	31
10.6 Checklist: release after go-live.....	31
11. Decisions and sections to complete.....	33
12. Document Sign-Off.....	34

1. Document Control

Date	Version	Author & Business Unit Info	Reviewed By & Business Unit Info	Approved By & Business Unit Info	Change History
04 October 2026	1.0	iorta TechNXT			Initial issue

2. Acronyms

Acronym / Symbol	Definition / Meaning
OOTB	Out of the box
Dev	Development environment
DR	Disaster recovery
SIT	System integration test
UAT	User acceptance test
Pre-Prod	Pre-production
CI	Continuous integration
CAB	Change advisory board
RACI	Responsible, Accountable, Consulted, Informed
GL	General ledger
TB	Trial balance
PITR	Point-in-time restore
PM	Project manager
PHT	Philippine time (UTC+8)
API	Application programming interface
XLSX	Excel workbook
DPA	Data Privacy Act of 2012
P1, P2	Incident priorities 1 and 2

3. Introduction

3.1 Purpose

This document answers the product owner's questions on how BrokerVerse OOTB is run across environments and taken into production for a broker:

- which environments exist, what each is for, who uses it, which data it holds, how it is hosted and sized, who may access it and how it is refreshed;
- how code and database scripts move from one environment to the next, with the checks, sign-offs and rollback at each step;
- how configuration moves between environments with the Go-Live Data Workbench, and what has to be entered or scripted separately;
- how the broker's own data flows in, how migrated business and new business are kept apart and in step;
- how the smoke test, the transaction reset, the final migration and the go-live lock fit together;
- the cutover runbook from 30 days before to 30 days after go-live, the RACI and the checklists for each promotion.

3.2 Sources and labels

Facts come from the code and the deployment files of the repository (IortaTechNXT-DEV/BD0I-00TB, branch brokerverse-platform, release tag v1.0.0): deploy/README.md, deploy/REFERENCE.md, deploy/RAILWAY.md, deploy/ec2/, .github/workflows/, backend/src/db/ (migrations and seeds), backend/scripts/ and backend/src/modules/data-load (the Go-Live Data Workbench). Text marked **Recommended** is iorta TechNXT practice, not something the code enforces. Text marked **Decision** is a product owner decision of 04 October 2026: the environment set by broker size, the temporary Pre-Prod, hosting per environment, one front-end build for all environments, a release pipeline with approval gates, a comparison report between environments and a data masking tool. Sections marked **Section to be completed with the release** describe deliveries still being built and are completed when they are released.

3.3 Related documents

Document	Use with this document
Data Migration and Cutover Plan	Data objects, control figures, mock loads, cutover checklist, go/no-go criteria, rollback of the data
Implementation Approach and Plan	Phases, timelines by broker size, project RACI, deliverables
UAT and Go-Live Acceptance Certificates	Form 1 (UAT Sign-off) and Form 2 (Go-live Acceptance) signed at the checkpoints of chapter 9
Hypercare Exit and Handover Certificate	End of hypercare and handover to support
Production Support Approach and Standards	Change types, CAB, release management, patching, regression testing
Release Notes Template	One copy per release or patch, attached to the CAB record
Business Continuity and Disaster Recovery Plan	Backups, restore procedures, recovery objectives
Architecture, Infrastructure, Security and Privacy	Hosting options A, B and C, sizing, CI/CD, secrets
Commercials and pricing workbook (BrokerVerse_Commercials_and_Pricing.xlsx) and Rate Card	Hosting price of each environment by broker size and provider
docs/onboarding/GO_LIVE_DATA_WORKBENCH.md	Go-Live Data Workbench: workbooks, validation, load, reconciliation
docs/onboarding/SMOKE_TEST_AND_RESET.md	Smoke test, transaction reset and go-live lock
docs/onboarding/GO_LIVE_DATA_SETUP.md	Step-by-step business set-up of a new installation

4. Environments

4.1 The five environment roles

	Dev	SIT	UAT	Pre-Prod	Production
Purpose	Development and integration of changes; first run of every migration; for small and medium brokers also the configuration build and the system integration test	Large brokers only: end-to-end test of a release with all roles; configuration build; mock loads	Business acceptance by the broker's key users; training; mock loads (all mock loads for small and medium brokers)	Temporary: dress rehearsal of the cutover; production smoke test; rehearsal of each major release after go-live	Live operations
Main users	iorta TechNXT developers and consultants	iorta TechNXT consultants and testers; migration lead	Broker key users, trainers, broker System Administrator; iorta TechNXT consultants	iorta TechNXT DevOps and migration lead; broker System Administrator, Accounting Manager and key users for the smoke test	All broker users; iorta TechNXT support by ticket
Data	Demo data (SEED_SAMPLE_DATA=true), synthetic UAT data (backend/scripts/uat-scenario.js) and the broker's configuration	Reference data, synthetic UAT data, the broker's configuration, mock-load data	Reference data, the broker's configuration, mock-load data from the broker's extracts	A restored production backup; before go-live it holds the production configuration and the rehearsal loads a fresh extract	Reference data, the broker's configuration, the final migration, then live business
Personal data	None (fictional)	Synthetic; client data during mock loads	Client data from the extracts	Client data; masked when used by people without production access	Client data
Release deployed	Any commit of a feature or fix branch	Release candidate tag	Release candidate tag that passed the system integration test	The tag to go to Production	Tag approved by the CAB

Decision: The broker's own data appears from the first mock load onwards. From that point the environment holding it is protected as Production is (section 4.6). A copy of production data used by people who have no production access is masked first, with the data masking tool (section 4.9). The privacy module anonymises one data subject at a time and is not used for bulk masking.

4.2 Environment sets by broker size

Decision: the environment set follows the broker size. Small and medium brokers have three standing environments: integration is done in the Dev instance and the release is pushed to UAT for testing, then to Production. Large brokers have four: SIT and UAT are separate. Pre-Prod is temporary for every size.

Role	Small broker	Medium broker	Large broker (and Enterprise)
Dev	Own Dev instance, small size: development, configuration build, integration and system	As small	Own Dev instance, small size: development and integration

Role	Small broker	Medium broker	Large broker (and Enterprise)
	integration test		
SIT	Not separate: done in Dev	Not separate: done in Dev	Own environment, medium size, separate from UAT
UAT	Own environment, small size	Own environment, small size	Own environment, medium size
Pre-Prod	Temporary	Temporary	Temporary
Production	Own environment, sized by tier	Own environment, sized by tier	Own environment with high availability; Enterprise adds a cross-region copy of the backups

The set is recorded in the Order Form (hosting fields) and in the environment sheet. A broker that hosts BrokerVerse itself provides the same set.

4.3 Pre-Prod lifecycle

Decision: Pre-Prod is temporary for every broker size.

Step	Rule
When it is created	For the go-live rehearsal (from about T-18 to the end of hypercare) and for each major release (about one month around the release)
Source	A backup of Production, restored at production topology and size. Before go-live, Production holds the production configuration promoted from UAT and no business; the rehearsal loads a fresh extract
Masking	Required before people without production access use it, with the data masking tool (section 4.9). People with production access may use the unmasked copy under the Production access rules
On restore	Own secrets, e-mail off, scheduler stopped, go-live lock handled as in section 4.8
Removal	After hypercare exit, or after the release: database, file store, backups and secrets deleted; the deletion recorded in the environment sheet
Hosting	Billed per month of use (section 3.5)

Each creation of Pre-Prod restores a Production backup in full and is recorded as a restore test of the Business Continuity and Disaster Recovery Plan.

Minor and patch releases do not use Pre-Prod: they are rehearsed in UAT (and in SIT for a large broker), and a snapshot of Production is taken before deploying.

4.4 Hosting by option

Every environment needs the same building blocks: the web build, the API (container from backend/Dockerfile or Node.js 22 under PM2), PostgreSQL 16 with the Asia/Manila time zone, a persistent file store at UPLOAD_DIR and a secret store. The hosting options are those of the Architecture document.

Environment	Option A: AWS Singapore	Option B: Azure Southeast Asia	Option C: Philippine partner or on-premise	Railway
Dev	One API task or one EC2 host (Docker Compose, or PM2 as in deploy/ec2), small RDS single-AZ	One VM or Container Apps replica, small Flexible Server without HA	One VM with Docker Compose	Suitable (deploy/RAILWAY.md)
SIT (large)	One API task (ECS)	One Container Apps	One API VM, one	Suitable for SIT and

Environment	Option A: AWS Singapore	Option B: Azure Southeast Asia	Option C: Philippine partner or on-premise	Railway
broker)	Fargate or EC2), RDS single-AZ, EFS	replica, Flexible Server without HA, Azure Files	PostgreSQL VM, NFS share	demo
UAT	As SIT, sized for the UAT performance test	As SIT	As SIT	Suitable when no performance test is needed
Pre-Prod (temporary)	Same topology as Production, restored from a Production backup: two API tasks behind a load balancer, RDS Multi-AZ, EFS, CloudFront	Same topology as Production	Same topology as Production	Not suitable
Production	Two or more API tasks, ALB, RDS Multi-AZ, EFS, CloudFront and WAF	Two or more replicas across zones, zone-redundant Flexible Server	Two API VMs, primary and standby PostgreSQL, redundant NFS	Not suitable: one API instance, volume attached to one instance

The demo deployment of the repository (.github/workflows/deploy-frontend.yml with deploy/ec2/deploy.sh: one PM2 process on port 8001 behind nginx) is a single instance without redundancy. It is fit for Dev, SIT or a demonstration, not for Production.

4.5 Sizing and hosting cost

Production sizes are those of the Architecture document and must be confirmed with the UAT performance test. Sizes for the other environments are those priced in the commercials and pricing workbook.

Item	Dev (every size)	UAT (small, medium)	SIT and UAT (large)	Pre-Prod	Production small / medium / large
API	1 x 1 vCPU, 2 GB	1 x 1 vCPU, 2 GB	1 x 1 vCPU, 2 GB (2 instances in UAT for the performance test)	As Production	2 x 1 vCPU, 2 GB / 2 x 1 vCPU, 2 GB / 3 to 4 x 2 vCPU, 4 GB
Database	2 vCPU, 4 GB, no standby	2 vCPU, 4 GB, no standby	2 vCPU, 8 GB, no standby	As Production	2 vCPU, 4 GB / 2 vCPU, 8 GB / 4 vCPU, 16 GB, each with standby
Database storage	20 GB	50 GB	100 GB	As Production	50 / 100 / 250 GB
File store	10 GB	25 GB	25 GB	As Production	25 / 60 / 150 GB (year 1)
Backups	Weekly	Daily, 14 days	Daily, 7 days (SIT), 14 days (UAT)	Daily while it exists	Daily with PITR, 35 days

Hosting price per month when iorta TechNXT hosts on AWS (Singapore), PHP excluding VAT, from the pricing workbook (Infrastructure sheet) and the Rate Card. Azure and the local partner are priced on the same sheet.

Broker size	Dev	SIT	UAT	Production	Standing set per month	Pre-Prod per month of use
Small	11,000.00	Not used	11,000.00	21,000.00	43,000.00	21,000.00
Medium	11,000.00	Not used	11,000.00	47,000.00	69,000.00	47,000.00
Large	11,000.00	23,000.00	23,000.00	86,000.00	143,000.00	86,000.00
Enterprise	11,000.00	23,000.00	23,000.00	199,000.00	256,000.00	169,000.00

Pre-Prod runs at production size and is billed only for the months it exists. The workbook budgets 2 months around go-live and 1 month for each major release. Hosting fees do not increase yearly; provider price changes are passed through at cost (Hosting and Infrastructure Services Agreement).

4.6 Access control

Environment	Application sign-in	Server, database and secret store	Rule
Dev	iorta TechNXT developers and consultants; demo users and UAT personas	iorta TechNXT developers	No client data, ever
SIT (large broker)	iorta TechNXT consultants and testers; UAT personas created by <code>uat-scenario.js</code> (<code>uat.maria.sales</code> and others)	iorta TechNXT DevOps	Client data only during mock loads; named accounts only
UAT	Broker key users and trainers with one role each; iorta TechNXT consultants	iorta TechNXT DevOps	Access list approved by the broker PM; two-step verification for system-admin, accounting and accounting-manager
Pre-Prod	Named people of the rehearsal and smoke test	iorta TechNXT DevOps; database access logged	Same controls as Production; masked first if anyone without production access uses it
Production	Broker users with their role; System Administrator role held by few people	iorta TechNXT DevOps under ticket and change approval	<code>deploy/README.md</code> section 5; Production Support change rules

Rules for every environment:

- Each environment has its own secrets. `JWT_SECRET` and `DATA_ENCRYPTION_KEY` differ between environments and are never reused from development (Architecture document).
- The BrokerVerse administrator password from `ADMIN_PASSWORD` is changed at the first sign-in, in every environment.
- The permissions of the Go-Live Data Workbench (`read:data-load`, `write:data-load`) are held by the System Administrator only.
- E-mail: in Dev, SIT and UAT, either keep "Send e-mails" off (Master > Configuration > Notification) or give test clients and insurers internal addresses, so that no client or insurer receives a test document.
- Payment gateways use sandbox credentials outside Production.

4.7 Refresh policy

Environment	When refreshed	How
Dev	At will	<code>npm run db:reset</code> (drops and recreates the schema, then seeds). Never run outside Dev
SIT (large broker)	Before each SIT cycle and before each mock load	Transaction reset (<code>CONFIRM_RESET=yes</code>) <code>npm run reset:transactions -- --</code>

Environment	When refreshed	How
		execute): masters and configuration stay, business records and opening balances go; then the next mock load. A full rebuild restores the configured baseline snapshot instead
UAT	Start of each UAT cycle; before each mock load	Configuration promoted from Dev (small, medium) or SIT (large) with the configuration workbook; transaction reset; migration workbook of the latest extract
Pre-Prod	Never refreshed in place	Created from a Production backup for the go-live rehearsal or a major release, then removed (section 4.3)
Production	Never refreshed	Only releases, CAB-approved configuration changes and live business

The transaction reset refuses to run while `goLive.locked` is on. Lower environments keep the lock off. Each refresh is recorded in the environment sheet with the date, the source and the person who ran it.

4.8 Pre-Prod: what must be identical to Production

Must be identical	Why
Release tag, backend image or commit, front-end build of the same tag	The rehearsal proves the exact code that goes live
Node.js 22, PostgreSQL 16 (same minor version), database time zone Asia/Manila	Reports, cut-off times and scheduled jobs use the database date
Topology: number of API instances, load balancer, file store type, CDN rules (/api/* not cached, single-page fallback)	Behaviour of the scheduler lock, file links and health checks
Non-secret environment values: <code>NODE_ENV=production</code> , <code>SEED_SAMPLE_DATA=false</code> , <code>SCHEDULER_ENABLED</code> , upload and import limits, token lifetimes, <code>LOG_LEVEL</code>	The production start-up check runs; limits behave the same
Configuration: everything in the configuration workbook and every on-screen item of section 6.3	The smoke test and rehearsal run on the production set-up
Document numbering series (prefix, pattern, reset rule)	Numbers look as they will in Production
Migration list of the database (<code>schema_migrations</code>)	No pending migration on either side before a release

What differs, by design: secrets (`JWT_SECRET`, `DATA_ENCRYPTION_KEY`, `ADMIN_PASSWORD`, database password, SMTP password), addresses (`CORS_ORIGINS`, `PUBLIC_BASE_URL`, `REACT_APP_BASE_URL`), payment gateway credentials (sandbox), e-mail recipients, and `APP_ENVIRONMENT`, which names the environment on the Instructions sheet of an exported workbook.

Pre-Prod is always restored from a Production backup. At each restore:

- Users with two-step verification sign in only if Pre-Prod uses the same `DATA_ENCRYPTION_KEY` as the backup. **Recommended:** keep Pre-Prod's own key and have an administrator turn two-step verification off for the rehearsal users, who enrol again (`deploy/REFERENCE.md`).

- The restored database has `goLive.locked` on. A System Administrator switches it off in Master > Configuration > Go-live before any reset; the change is recorded in the audit trail. This is never done in Production.
- Turn "Send e-mails" off and stop the scheduler (`SCHEDULER_ENABLED=false`) before the first start, so that reminders and renewal notices are not sent to real clients from Pre-Prod.
- Mask the copy with the data masking tool before anyone without production access signs in (section 4.9).

4.9 Data masking

Decision: a data masking tool is required. It masks the personal data of a restored production copy before it is used by people who have no access to Production, for example a Pre-Prod copy for a release rehearsal with testers, or a training copy.

Section to be completed with the release. The tool, what it masks, how it is run and how a masked copy is checked are described when the tool is released.

5. Moving code and database scripts

5.1 Git branching and release tagging

Branch or tag	Use
brokerverse-platform	Release branch of BrokerVerse OOTB; what is deployed comes from here
feature/<ticket>-<short-name>	Recommended for changes; merged by pull request
fix/<ticket>-<short-name>	Fixes, started from the release tag running in Production (Production Support, code fix standards)
vX.Y.Z tag	One per release: patch (1.0.z), minor (1.y.0), major (x.0.0), as in the Release Notes Template. v1.0.0 is the first release

Every change reaches brokerverse-platform through a pull request reviewed by an engineer other than the author. A fix merged to the release branch is also merged into the main development line so it is not lost in the next release.

5.2 What CI checks

.github/workflows/ci.yml runs on every pull request:

Job	Steps
Backend	Node.js 22; npm ci; npm run lint (ESLint on src, test, scripts); npm test (vitest) against a postgres:16 service database brokerverse_test
Frontend	Node.js 22; npm ci --legacy-peer-deps; npx craco test --watchAll=false

The backend tests build the schema from the migrations, so a migration that does not apply on an empty database fails CI.

.github/workflows/deploy-frontend.yml (CI/CD Pipeline) runs on a push to brokerverse-platform: it builds the front end, publishes it to S3 and CloudFront, and reloads the API on the EC2 host through deploy/ec2/deploy.sh, which waits for /api/health to answer 200. Its lint and test jobs are commented out, so a push deploys without tests, and it serves one environment.

Decision: the manual deployment is replaced by a release pipeline built to current practice: changes arrive on brokerverse-platform only by pull request with a green CI run, the tests run before any deployment, and each environment has its own deployment step started from a tag, with an approval gate per environment. Until the pipeline is released, each deployment to SIT, UAT, Pre-Prod and Production is started by the DevOps lead from the approved tag.

5.3 Release pipeline and approval gates

Section to be completed with the release. The pipeline stages, the approval gate and approver of each environment, and how a deployment is recorded are described when the pipeline is released.

5.4 Build once, promote the same artefact

Part	Built	Promoted
Backend	Recommended: one container image from backend/Dockerfile per tag, with GIT_COMMIT set so GET /api/version	The same image digest is deployed to SIT, UAT, Pre-Prod and Production. Only environment variables differ

Part	Built	Promoted
	shows it; pushed to the registry (ECR, Azure Container Registry or the partner registry)	
Front end	Today: <code>npm run build</code> of the same tag, with <code>REACT_APP_BASE_URL</code> compiled into the build, so each environment gets its own build of the same commit. Decision: one build per tag, with the environment values read at runtime (section 5.5)	Today: rebuilt per environment from the tag, the build log and file hashes kept with the release record. After the change: the same build promoted to every environment
Database	Migration files inside the image (backend/src/db/migrations)	Applied by the image itself on start (section 5.6)
Configuration	Configuration workbook (chapter 6)	Downloaded from the source environment, loaded into the target

Today the EC2 script deploys a Git checkout (`npm ci --omit=dev`) rather than an image. Deploying a tag, not a branch head, gives the same result: every environment runs the same commit.

5.5 Front-end build and runtime configuration

Decision: one front-end build is promoted through all environments, and the API address and the other environment values are read at runtime instead of being compiled into the build.

Section to be completed with the release. How the runtime values are supplied in each environment, and how the build is checked before promotion, are described when the change is released.

5.6 Schema migrations

- Migrations are numbered SQL files in `backend/src/db/migrations` (98 files in v1.0.0, from `0001_core.sql` to `0245_user_profile_details.sql`).
- On every API start, `backend/src/db/migrate.js` takes a PostgreSQL advisory lock (other instances wait up to 10 minutes, `MIGRATION_LOCK_TIMEOUT_MS`), applies each file not yet recorded in `schema_migrations` in file-name order, each in its own transaction, and records it. A failing migration is rolled back and stops the start-up. `/api/health` answers 503 until the migrations and the seed have finished.
- By command: `npm run migrate` applies the pending migrations without starting the API (same lock). `GET /api/health` and `GET /api/version` report pendingMigrations.
- `npm run db:reset (migrate.js --reset)` drops the whole schema. It is for Dev only.

Rules (project convention and **Recommended** practice):

1. A migration file is never edited after it has left Dev. The runner identifies a migration by its file name only: an edited file is not run again where it was already applied, so environments would differ silently. A correction is a new migration with a higher number.
2. Forward only. Migrations add tables, columns and indexes; existing data is changed only by a reviewed migration or a data correction under change

control. There are no down-migrations: the previous release runs on the newer schema.

3. Every migration runs in Dev, then SIT (large broker) and UAT, in that order, before it reaches Production. For a major release it also runs in Pre-Prod, the rehearsal on a copy of production data.
4. Before each Production deployment that contains a migration, take a manual database snapshot (the restore point), deploy one API instance first, wait for `/api/health` ready with `pendingMigrations: 0`, then roll the other instances.
5. A new table must be classified in `backend/scripts/lib/table-classification.js` (transaction, system, or master and configuration). A test fails otherwise, and the transaction reset refuses to run on a database with an unclassified table.

5.7 Reference seeds

`backend/src/db/seed.js` runs after the migrations on every start, and by command with `npm run seed`. It is idempotent: rows are inserted by natural key or fixed id, existing rows and administrator edits are kept.

Part	Content	When
In code	Roles, permissions and role grants; the BrokerVerse administrator	Always
<code>settings.json</code> , <code>jobs.json</code>	Configuration keys of Master > Configuration; scheduled jobs	Always
<code>seeds/*.sql</code>	Reference data: geography, currencies, banks, insurers, products, covers, chart of accounts, product templates and motor tariff, report catalogue, security and direct-bill configuration	Always
<code>seeds/sample/*.sql</code>	Demo leads, clients, policies, claims, users	Only with <code>SEED_SAMPLE_DATA</code> on (default off with <code>NODE_ENV=production</code>)

A new setting has its code fallback equal to the seeded value; `npm run check:settings` checks it. A database started with sample data by mistake is cleaned with `npm run purge:sample` before real business is entered.

5.8 Environment variables and secrets

Secrets are never in the repository, a ticket or a chat: they live in the secret store of each environment (AWS Secrets Manager or SSM Parameter Store, Azure Key Vault, HashiCorp Vault, or Railway variables). `deploy/backend.env.example` holds the names with placeholders only.

Variable	Dev	SIT / UAT	Pre-Prod	Production
<code>NODE_ENV</code>	development	production	production	production
<code>DATABASE_URL</code>	Local	Own database	Own database	Own database, TLS
<code>JWT_SECRET</code> , <code>DATA_ENCRYPTION_KEY</code>	Own values	Own values per environment	Own values	Own values, escrow copy under dual control
<code>ADMIN_PASSWORD</code>	Own	Own	Own	Own, changed at first sign-in
<code>CORS_ORIGINS</code> , <code>PUBLIC_BASE_URL</code>	Local addresses	Environment addresses	Environment addresses	Production addresses

Variable	Dev	SIT / UAT	Pre-Prod	Production
SEED_SAMPLE_DATA	true	false	false	false
SCHEDULER_ENABLED	true	true	false until the rehearsal needs it	true
SMTP_URL	Test mailbox or none	Test mailbox	Test mailbox	Production mailbox
Payment gateway (<prefix>_SECRET_KEY and others)	None	Sandbox	Sandbox	Live
APP_ENVIRONMENT	Dev	SIT / UAT	Pre-Prod	Production
REACT_APP_BASE_URL (front-end build today; read at runtime after the change of section 5.5)	Local API	Environment API	Environment API	Production API

With `NODE_ENV=production` the API refuses to start when `JWT_SECRET` or `DATA_ENCRYPTION_KEY` is missing, short or a placeholder, when the two are equal, when `CORS_ORIGINS` is `*`, or when `PUBLIC_BASE_URL` is `localhost` (`deploy/REFERENCE.md`).

5.9 Promotion steps

Step	Entry criteria	Activities	Exit criteria	Sign-off
System integration test in Dev (small and medium broker)	Pull requests merged with green CI; release candidate tagged	Deploy the tag to Dev; migrations and seed apply on start; smoke test (<code>deploy/README.md</code> section 4); <code>uat-scenario.js</code> on a fresh database; end-to-end plan; configuration built	Smoke test passed; <code>uat-scenario.js</code> all steps passed; no open P1 or P2 defect	iorta TechNXT delivery lead (SIT exit report)
Dev to SIT (large broker)	Pull requests merged with green CI; release candidate tagged	Deploy the tag; migrations and seed apply on start; smoke test; <code>uat-scenario.js</code> on a fresh database; end-to-end plan; configuration built	Smoke test passed; <code>uat-scenario.js</code> all steps passed; no open P1 or P2 defect from SIT	iorta TechNXT delivery lead (SIT exit report)
Dev or SIT to UAT	SIT exit report; release notes drafted; configuration workbook downloaded from Dev (small, medium) or SIT (large)	Deploy the same tag; promote the configuration (chapter 6); transaction reset; load the latest mock migration; key users run the UAT scripts	UAT scripts passed or accepted with a workaround; mock load reconciled	Broker process owners and sponsor: UAT Sign-off Certificate (Form 1)
UAT to Production (release and configuration)	UAT Sign-off; configuration frozen; CAB approval of the release	Deploy the same tag to Production; production secrets; promote the configuration from UAT; on-screen items; technical smoke test	Configuration comparison UAT against Production shows no change	iorta TechNXT DevOps lead; broker System Administrator
Pre-Prod rehearsal	Production configured; Production backup taken	Create Pre-Prod from the backup (section 4.3); cutover rehearsal: smoke test, reset, migration load, reconciliation, timings	Rehearsal inside the cutover window; reconciliation agrees	iorta TechNXT DevOps lead and migration lead; broker PM
Go-live in Production	Go/no-go checkpoint 1 passed; CAB approval; restore	Final migration at cutover; reconciliation; go-live lock	Final reconciliation signed; go decision	Steering committee; Accounting Manager for the

Step	Entry criteria	Activities	Exit criteria	Sign-off
	point taken			reconciliation; Go-live Acceptance Certificate (Form 2)

After go-live, every release follows the same path to UAT. A major release is then rehearsed in a Pre-Prod created from a Production backup, which is removed after the release. A minor or patch release goes from UAT to Production with a manual snapshot of Production taken before deploying.

5.10 Change and release management

Item	Rule (Production Support Approach and Standards)
Change types	Standard (pre-approved, done by the broker's System Administrator), Normal (release, tax rate, posting rule, numbering series, data correction script: CAB approval), Emergency (P1 fix or security patch: approved by the support manager and the broker's IT head, reviewed at the next CAB)
CAB	The broker's change owner and the iorta TechNXT support manager; approves Normal changes with impact, test evidence and rollback
Release types	Planned release (monthly, when there are fixes or changes); emergency release
Notice	Release notes to the broker's System Administrator and IT head at least 5 business days before deployment
Window	Maintenance window, default Saturday 20:00 to Sunday 06:00 PHT
Evidence	Regression of Production Support chapter 19: backend and front-end tests, uat-scenario.js on a fresh database, the UAT scripts of the changed area, the document and report sweep for print changes
After deploying	Smoke test; GET /api/version shows the expected commit and no pending migration

The release notes use the Release Notes Template: release identification (tag, commit, previous release, window, downtime, CAB reference), changes by module, migrations included, actions an administrator must take by hand (including on-screen configuration items), known issues, and rollback.

5.11 Rollback

What	How
Front end	Re-run the build of the previous tag, or restore the previous S3 object versions (bucket versioning on), then invalidate CloudFront
Backend	Redeploy the previous image or tag. Migrations only add, so the previous version runs on the newer schema
Database	Restore the pre-release snapshot only when data written by the new release must be undone; transactions entered since the snapshot are re-entered
Configuration	Load the configuration workbook downloaded before the change (Current data). The workbook updates and creates but never deletes: records created by the change are set inactive on their screen
Go-live data	Before go-live: restore the snapshot taken before the loads. After go-live: rollback window and procedure of the Data Migration and Cutover Plan

6. Moving configuration

6.1 The configuration workbook

The configuration kit of Master > Go-Live Data Load (GoLive_Configuration_Workbook.xlsx) carries what is needed to run new business: company, settings, location masters, branches, departments, hierarchy, designations, users, currencies, exchange rates, chart of accounts, banks, bank accounts, signatories, transaction codes, write-off reasons, insurers, lines of business, products, policy types, covers, vehicle brands, models, variants and vehicles, commission rate matrix, premium taxes and charges, LGU tax rates, authority limits and document numbering.

Each sheet is matched on a natural key (code, username, setting key, account code, series code). Loading the same or a corrected workbook again updates and never duplicates; rows equal to the stored data are reported as unchanged and not written.

6.2 Promotion procedure

1. **Source environment.** Master > Go-Live Data Load > **Download Template > Current data** (GET /api/data-load/kits/configuration/template?prefill=true). The export is recorded in the audit trail. The file name carries the date; the Instructions sheet names the environment (APP_ENVIRONMENT).
2. **Review before loading.** Edit the workbook for the target where needed:
 - **Users:** delete test personas (uat.*), trainers' test accounts and anyone who must not have access to the target. Passwords are never in the workbook: each new user gets a temporary password shown once to the administrator after the load.
 - **Numbering: Next Number** is exported from the source counters, which testing has advanced. For Production, set it to the last number used in the old system plus one. It cannot go backwards once loaded.
 - **Settings:** check golive.cutover_date and any environment-specific setting. Settings of other screens, settings under second-user approval, secret-like keys and golive.locked are never exported or loaded.
3. **Target environment.** Master > Go-Live Data Load > upload the workbook (POST /api/data-load/batches, kit configuration). Validation is a dry run: all sheets run in load order inside one database transaction that is rolled back.
4. **Read the result.** Per sheet: rows read, valid, with errors, and new, changed, unchanged or for approval.
5. **Fix.** Download errors (GET /api/data-load/batches/:id/errors): only the rows in error, with an Errors column. Correct and upload again.
6. **Load** (POST /api/data-load/batches/:id/load). For the configuration workbook, **Load valid rows only** is ticked by default. The load is one transaction, written to the audit trail, and the administrator is notified.
7. **Approve.** Authority limits load as proposals: another administrator approves them on Master > Generals > User Management > Authority Matrix in the target.

8. **Complete the on-screen items** of section 6.3 and record them on the promotion checklist.

6.3 Not in the workbook

These items are entered on their screen in each environment, or applied by a scripted procedure agreed with the DevOps lead. The list is the one on the Instructions sheet of the workbook.

Item	Where	How it moves
Roles and their permissions	Master > Generals > User Management > Role	Roles and permissions are seeded; changes repeated on screen
Segregation of duties rules, delegations, access reviews	Master > Generals > User Management	On screen
Approval of the loaded authority limits	Master > Generals > User Management > Authority Matrix	Second administrator approves in the target
Tax codes (VAT, withholding, BIR ATC)	Master > Finance > Taxation	On screen, confirmed by the broker's tax adviser
Account determination and posting rules	Master > Finance > Account Determination / Posting Rules / Configuration Approvals	On screen, proposed by one user and approved by another
Bank statement formats, bank transaction types, insurer statement formats, close checklist	Master > Finance	On screen
Product templates, rating, acceptance rules, motor tariff, documents	Product Configurator > Product Templates	On screen
Package bundles and insurer rate tables	Master > Packaged Products	On screen
Payment gateway credentials	Master > Finance > Payment Gateways; secret store	Secrets set per environment in the secret store; sandbox outside Production
Application name, logo, colours, display currency, language	Master > System Settings	On screen
Company logo and letterhead images	Master > Generals > Organization > Company	Uploaded on screen
Scheduled jobs	Master > Configuration > Schedules	On screen; switched on at go-live
Fiscal years and accounting periods	Accounts > Period End > Period Management	On screen; periods before the cutover date closed
Remittance masters, reinsurance treaties and reinsurers, incentive programmes	Master > Finance > Remittance Master; Reinsurance; Incentive	On screen
Referrers and agents with their commission accounts	Commission > Agents/Referrer Accounts	On screen
Petty cash funds	Accounts > Petty Cash > Initiate	On screen
Users' passwords	Master > Generals > User Management > User	Temporary password at load; named administrators with provision-users.js
Go-live lock (goLive.Locked)	Master > Configuration > Go-live	Switched on in Production only, after the migration is signed off
Environment variables and secrets	Secret store	Per environment, never copied

Recommended: Keep a promotion checklist with one line per on-screen item, the value set in UAT and the tick in each target environment. It is the evidence for item 5 of the cutover checklist (configuration copied and compared item by item).

6.4 Configuration freeze

- **Freeze for UAT:** configuration is frozen at the end of the system integration test, in Dev (small and medium broker) or SIT (large broker) (Implementation Approach). Changes after that are logged and repeated in UAT.
- **Freeze for cutover:** from UAT sign-off (about T-21), no configuration change is made unless it is a Normal change approved by the CAB. An approved change is made in UAT first and promoted to Production with the workbook or the on-screen procedure (and repeated in Pre-Prod while it exists), never made directly in Production.
- **After go-live:** configuration changes in Production follow the change types of Production Support. Settings changes are recorded in the audit trail with the old and new value.

6.5 Comparing environments

The validation of a **Current data** workbook is also the comparison report between two environments:

1. Download **Current data** from environment A.
2. Upload it into environment B and stop after validation (do not load).
3. The result per sheet gives new, changed and unchanged rows. Two environments with the same configuration show every row unchanged and nothing new or changed.

Run it UAT against Production after the configuration is promoted, and Production against Pre-Prod after the rehearsal, to show that no configuration was changed during the rehearsal. The validation result is kept in the load history (status validated or failed) as evidence. On-screen items are compared with the promotion checklist.

6.6 Comparison report

Decision: a comparison report between two environments is added, listing the differences row by row. Until it is released, the detail is found by comparing the two **Current data** workbooks in Excel.

Section to be completed with the release. What the report compares, how it is run and how its result is kept as evidence are described when the report is released.

7. Client data flow

7.1 The two workbooks the broker fills

Workbook	Content	Filled by	Used in
Configuration workbook (masters for new business)	Organisation, users, insurers, products, chart of accounts, banks, commission rates, taxes and charges, numbering, settings	Broker process owners with iorta TechNXT consultants, from the discovery decisions	Built in Dev (small, medium) or SIT (large), then promoted to UAT and Production; Pre-Prod receives it with the Production backup
Migration workbook (existing business)	Clients, in-force policies with legacy numbers, open premium receivables, open claims, GL opening balances	Broker data owners, from extracts of the old system; Accounting Manager for the financial sheets	Mock loads in UAT (small, medium) or SIT and UAT (large); rehearsal in Pre-Prod; final load in Production

Blank workbooks come from Master > Go-Live Data Load (**Blank template**) or from docs/package/05_Delivery/Upload_Templates (generated by backend/scripts/build-upload-templates.js). Each has an Instructions sheet, a Lists sheet with the allowed values and one sheet per object in load order.

7.2 Mock loads and the final load

Load	Environment	Data	Exit
Mock 1	UAT (small, medium); SIT (large)	Full extract at a recent date	All sheets validate; failures listed and assigned for cleansing
Mock 2	UAT (small, medium); SIT or UAT (large)	Fresh extract after cleansing	Failures below 1% of rows and explained; control figures reconcile
Mock 3 (medium and large)	UAT	Fresh extract at a month-end	All control figures reconcile; UAT on migrated data passes
Rehearsal	Pre-Prod (every size)	Fresh extract, with the cutover timetable	Done inside the cutover window; reconciliation signed
Final load	Production, at cutover	Extract at the close of the day before the cutover date	Reconciliation signed by the data owners and the Accounting Manager

Each mock load follows the same cycle:

1. Set `golive.cutover_date` for the mock (the first day after the extract date).
2. Transaction reset of the environment (the earlier mock goes, masters and configuration stay).
3. Upload the migration workbook; read the errors; fix in the old system or the workbook; upload again.
4. Load (for the migration workbook **Load valid rows only** is not ticked by default: a clean, complete load is the goal).
5. Download the reconciliation (GET `/api/data-load/batches/:id/reconciliation`) and compare it with the control figures of the old system.

6. Record the timings and the sign-off in the reconciliation report of the Data Migration and Cutover Plan.

The reconciliation gives, per sheet, the rows and totals of the workbook (gross and net premium, sum insured, open balance, claim estimate, debits and credits) and the records and totals in BrokerVerse after the load, plus two checks: TB debits equal credits; the premiums receivable control account opening balance equals the total open balance of the open items.

7.3 Keeping migrated and new business apart and in step

Topic	How BrokerVerse handles it
Flags	Migrated clients carry source = 'go-live-migration' and load_batch_id; policies doc.source = 'go-live-migration' and load_batch_id; open items are receivables with source = 'opening', go_live_date and load_batch_id; open claims details.source = 'go-live-migration' and load_batch_id
Legacy numbers	Migrated records keep the client code, policy number and claim number of the old system
No postings	A migrated policy creates no bill, booking journal or commission accrual; an open item posts no journal; an open claim sends no acknowledgement and posts nothing. The GL opening balances carry the money
Opening balances	The TB of the old system at the close of the day before the cutover date (cutover minus one day). Debits equal credits; the sheet loads all or nothing (the rows in error show their errors, the others are held, one message for the sheet); a row with no debit or credit is ignored; loading again replaces the earlier load. The fiscal year must have no journal posted before the cutover
Dates	Policy issue dates, claim loss and reported dates and client birth dates must be before the cutover date. Policies must be in force at cutover. Every new transaction is dated on or after the cutover date
Numbering	New business takes the next number of its Document Numbering series. The Numbering sheet sets the next number above the migrated range. Both workbooks refuse a collision: a series whose next number would issue a number a migrated record already has, or a legacy number in the range a series has still to issue
Renewals	Migrated in-force policies are ordinary active policies: the renewal queue picks them up as they near expiry. The renewal is a new BrokerVerse policy with its own number, bill and journals
Receipts	Open items age, are chased by the collection reminders and are paid with normal official receipts
Amounts due to insurers and referrers	In the opening balances (Due to Insurers, Commission Payable) and paid with payment vouchers (Accounts > Disbursement), not through the remittance run
Executive dashboard	Premium written and new business exclude migrated policies (doc.source = 'go-live-migration' or a load_batch_id); migrated policies count as policies in force
Month-end checks	The check "policies without accounting" ignores migrated policies, which were billed by the old system
Lists and reports	Policy lists filter on the source (GET /api/policies? source=go-live-migration). The Production Register

Topic	How BrokerVerse handles it
	shows a Source column (Migrated or BrokerVerse), counts migrated policies in its summary and groups by source with the Report Criteria Source, so business written in BrokerVerse is reported apart from the migrated book. The executive dashboard leaves migrated policies out of premium written and new business
Audit	load_batch_id ties every migrated record to its batch; the load history keeps who loaded what and when

After the go-live lock, the migration workbook is refused. A record missed by the final load is entered on its screen as an exception, agreed with the Accounting Manager.

8. Smoke test and go-live

8.1 Order of events

1. **Smoke test, preferably in Pre-Prod** with the production configuration: a quotation to policy, bill, receipt, remittance, commission and journals, a claim and a renewal (deploy/README.md section 4 and docs/onboarding/SMOKE_TEST_AND_RESET.md).
2. **If the smoke test must run in Production** (for example to prove the production e-mail, printer or payment gateway set-up): agree it in the cutover plan, take a database snapshot, use clearly named test data (a test branch such as TST, a test insurer and clients named ZZ TEST . . .), and send nothing to real clients or insurers.
3. **Transaction reset** after a smoke test in Production (or before each rehearsal in Pre-Prod):
 - stop the API instances; dry run `CONFIRM_RESET=yes npm run reset:transactions;`
 - execute with `CONFIRM_RESET=yes RESET_ACTOR=<username> npm run reset:transactions -- --execute`, adding `--purge-files` to delete the files of the test transactions;
 - masters, configuration, users and settings are kept; transaction document series restart at the next number the Numbering sheet set (else their start number); the TB is empty and every period is open again; the reset is recorded in the audit trail;
 - masters created only for the test (test branch, insurer, product) are set inactive or deleted on their screen.
4. **Final migration load** of the migration workbook in Production at cutover, then the reconciliation and its sign-off.
5. **Go-live lock:** a System Administrator switches on `goLive.locked` (Master > Configuration > Go-live). From then on the transaction reset refuses to run (exit code 3) and the migration workbook is refused (upload, validation and load). The configuration workbook stays available for new masters. There is no option of the command that bypasses the lock; switching it off is audited and is never done on the live Production database.
6. **Hypercare** starts on the go-live day.

8.2 Hypercare

Item	Rule
Length	Default 2 weeks (small), 3 weeks (medium), 4 to 6 weeks (large); extended to cover the first month-end close
Rhythm	Daily check-in in the first week, then weekly
Rollback window	Until the end of the first business week; afterwards fix forward
Evidence during the window	Transactions entered in BrokerVerse listed daily (Receipts Register, Production Register, Journal) so a rollback could be carried out
Exit	Hypercare Exit and Handover Certificate when the criteria of Production Support chapter 16 are met: no open P1 or P2,

Item	Rule
	first month-end close approved, configuration baseline recorded (a Current data configuration workbook of Production), environment sheet current

9. Cutover runbook

9.1 Conventions

T is the go-live day: the first day of transactions in BrokerVerse and the value of `golive.cutover_date`. The example assumes T is a Monday. Days are calendar days. Hours are **indicative** and are replaced with the times measured in the rehearsal. Checkpoints GNG-1 and GNG-2 are the go/no-go checkpoints 1 and 2 of the Data Migration and Cutover Plan.

9.2 T-30 to T-1

When	Step	Owner	Evidence
T-30	Production provisioned at production topology; secrets created in the secret store; backups running; one restore test done	iorta TechNXT DevOps lead	Environment sheet; restore test record
T-30	Release candidate tag fixed; from now only P1 and P2 fixes enter it	iorta TechNXT release manager	Tag and release notes draft
T-30	Go-live date confirmed; <code>golive.cutover_date</code> set in UAT for the last mock load	Broker PM, Accounting Manager	Steering committee minutes
T-28	Last mock load in UAT on a fresh extract; reconciliation compared with the control figures	iorta TechNXT migration lead; broker data owners	Reconciliation workbook; sign-off
T-21	UAT completed and signed: UAT Sign-off Certificate (Form 1)	Broker process owners and sponsor	Form 1
T-21	Configuration freeze declared; changes only by CAB	Broker PM; iorta TechNXT delivery lead	Freeze notice
T-20	Release deployed to Production; technical smoke test of <code>deploy/README.md</code> section 4; administrator password changed; two-step verification on for the required roles	iorta TechNXT DevOps lead; broker System Administrator	Smoke test record
T-19	Configuration promoted from UAT to Production: Users sheet without test personas, Numbering sheet set from the old system's last numbers plus one; authority limits approved; on-screen items entered; comparison UAT against Production shows no change	Broker System Administrator; iorta TechNXT consultant	Load history; validation result; promotion checklist

When	Step	Owner	Evidence
T-18	Production backup taken; Pre-Prod created from it at production topology (section 4.3); own secrets, e-mail off, scheduler stopped; masked first if anyone without production access takes part	iorta TechNXT DevOps lead	Environment sheet; restore record (counts as a restore test)
T-17 to T-15	Cutover rehearsal in Pre-Prod: smoke test, transaction reset, migration load of a fresh extract, reconciliation, go-live lock switched on and off again; each step timed	iorta TechNXT migration lead; broker Accounting Manager and key users	Rehearsal log with timings; reconciliation signed
T-14	CAB approves the Production deployment and the cutover plan	Broker change owner; iorta TechNXT support manager	CAB record
T-14	GNG-1: UAT signed, rehearsal reconciled, training on track, configuration ready in Production	Steering committee	Minutes
T-12	Live payment gateway credentials and SMTP in the secret store; any configuration change found in the rehearsal made in UAT and promoted to Production	Broker System Administrator; iorta TechNXT DevOps lead	Promotion checklist; CAB record of any change
T-11	Comparison Production against Pre-Prod shows no configuration change other than those approved; tax codes, premium taxes and LGU rates confirmed by the tax adviser	iorta TechNXT consultant; broker Accounting Manager	Validation result; adviser confirmation
T-10	Smoke test in Production only if not done in Pre-Prod: snapshot, test data, then transaction reset and check that Leads, Clients, Policies and the TB are empty	iorta TechNXT DevOps lead; broker key users	Snapshot id; reset audit entry
T-7	Users signed in once in Production; communication sent to users, insurers and banks; hypercare roster published	Broker PM; broker System Administrator	Sign-in history; communication log
T-5	Masters reconciled in Production; document numbering checked on the screen preview; fiscal year and periods set, periods before T closed	Broker System Administrator; Accounting	Cutover checklist items 3 to 12
T-3 (Friday) 17:00	Old system frozen for new transactions; last receipts and vouchers of	Broker PM	Freeze notice

When	Step	Owner	Evidence
	the day posted there		
T-3 (Friday) 18:00 to 22:00	Day-end in the old system; extracts of clients, in-force policies, open receivables, open claims and TB; control figures produced and signed	Broker data owners; Accounting Manager	Signed control figures
T-2 (Saturday) 08:00	Production snapshot before the loads; golive.cutover_date = T checked in Production	iorta TechNXT DevOps lead	Snapshot id
T-2 (Saturday) 09:00	Migration workbook uploaded and validated; errors downloaded and fixed; validated again	iorta TechNXT migration lead; broker data owners	Load history
T-2 (Saturday) 13:00	Migration workbook loaded	iorta TechNXT migration lead	Batch status loaded
T-2 (Saturday) 14:00 to 18:00	Reconciliation downloaded and compared with the control figures; differences explained	Migration lead; Accounting Manager	Reconciliation workbook
T-1 (Sunday) 09:00	Key user sample checks (20 policies per line, 20 open items, TB line by line)	Broker key users; Accounting Manager	Sample check sheet
T-1 (Sunday) 12:00	Reconciliation report signed	Broker data owners; Accounting Manager	Signed report
T-1 (Sunday) 14:00	GNG-2 (final): criteria of the Data Migration and Cutover Plan	Steering committee	Minutes; go decision
T-1 (Sunday) 15:00	golive.locked switched on; snapshot of the go-live baseline	Broker System Administrator; iorta TechNXT DevOps lead	Audit entry; snapshot id
T-1 (Sunday) 16:00	Scheduled jobs and "Send e-mails" switched on; old system set to read-only	Broker System Administrator; iorta TechNXT DevOps lead	Schedules screen; E-mail Outbox

9.3 T to T+30

When	Step	Owner	Evidence
T 07:30	Health checks (/api/health ready, /api/version shows the tag); floor support in place	iorta TechNXT DevOps lead; hypercare team	Monitoring
T	First policies, receipts and bank imports watched; daily list of BrokerVerse transactions for the rollback window	Hypercare team; broker key users	Daily list
T 17:00	Daily hypercare call: issues, decisions, rollback status	Broker PM; iorta TechNXT PM	Hypercare log
T+1 to T+4	Daily check-ins; first remittance to insurers; first bank statement per account	Accounting; hypercare team	Remittance reference; bank reconciliation

When	Step	Owner	Evidence
T+4 (Friday)	Rollback window ends; decision recorded (continue, or roll back as in the Data Migration and Cutover Plan)	Steering committee	Minutes
T+7	Go-live criteria checked; Go-live Acceptance Certificate (Form 2) signed	Broker sponsor; iorta TechNXT PM	Form 2
T+7 to T+30	Weekly hypercare meetings; known issues and workarounds recorded	iorta TechNXT PM; broker PM	Hypercare log; knowledge base
T+7 to T+30	Pre-Prod kept to rehearse hypercare fixes on the go-live data	iorta TechNXT DevOps lead	Environment sheet
First month-end	Month-End Close prepared by Accounting and approved by the Accounting Manager; TB compared with the opening balances plus the month's activity	Accounting; Accounting Manager	Approved Month-End Close run
T+30 (or after the first close)	Hypercare exit: configuration baseline (Current data workbook of Production) and environment sheet handed over; Hypercare Exit and Handover Certificate	iorta TechNXT PM and support manager; broker IT head	Certificate
After hypercare exit	Pre-Prod removed: database, file store, backups and secrets deleted; hosting of Pre-Prod stops	iorta TechNXT DevOps lead	Environment sheet; deletion record

10. RACI and checklists

10.1 RACI for environment and release activities

R responsible, A accountable, C consulted, I informed. iorta TechNXT roles: DevOps lead (Ops), delivery team with the migration lead (Dlv), release and support manager (Sup). Broker roles: project manager (PM), System Administrator (SA), Accounting Manager (AM), key users and process owners (KU), IT head and change owner (IT).

Activity	Ops	Dlv	Sup	PM	SA	AM	KU	IT
Provision and size environments	A, R	C	I	I	I			C
Secrets and secret store	A, R		I					C
Branching, CI, release tag	A, R	C	C					I
Deploy to Dev and SIT	A, R	C						
System integration test (Dev or SIT) and SIT exit report	C	A, R	I	I				
Promote configuration from Dev or SIT to UAT	C	A, R		I	C	C		
UAT and UAT sign-off	C	R		A	R	R	R	I
Mock loads and reconciliation	C	A, R		I	C	R	R	
Configuration freeze		R	C	A	C	C	I	I
CAB approval of the release	C	C	R	I				A
Deploy to Production	A, R	C	C	I	I			C
Create, mask and remove Pre-Prod	A, R	C	I	I	C			C
Promote configuration to Production; on-screen items	C	R		I	A, R	C		
Smoke test and transaction reset	R	R		I	A	C	R	I
Final migration load	C	R		I	C	A	R	
Go-live lock	C	C		I	A, R	C		I
Go/no-go checkpoints	C	R	C	A	C	C	C	C
Rollback decision	R	C	C	A	C	C	I	C
Release after go-live	R	C	A	I	C		C	C

The steering committee holds the go/no-go and rollback decisions; the PM column shows the broker's accountable member of it.

10.2 Checklist: system integration test (Dev or SIT)

#	Check
1	Pull requests merged; CI green on the last one; release candidate tagged
2	New tables classified in table-classification.js; upload templates regenerated if an importer changed
3	Release notes drafted, with migrations and manual actions listed
4	Deployed to Dev (small, medium) or SIT (large); /api/health ready with pendingMigrations: 0; /api/version shows the tag

#	Check
5	Smoke test of deploy/README.md section 4 passed
6	uat-scenario.js on a fresh database: all steps passed

10.3 Checklist: Dev or SIT to UAT

#	Check
1	SIT exit report: end-to-end flows passed, no open P1 or P2
2	Same tag deployed; migrations applied on start
3	Configuration workbook (Current data) of Dev or SIT reviewed (users, numbering) and loaded; authority limits approved
4	On-screen items entered and ticked on the promotion checklist
5	UAT access list approved; two-step verification on for the required roles; "Send e-mails" off or internal addresses only
6	Transaction reset; latest mock migration loaded and reconciled

10.4 Checklist: UAT to Production and the Pre-Prod rehearsal

#	Check
1	UAT Sign-off Certificate (Form 1) signed; configuration freeze declared; CAB approval
2	Production secrets set; SEED_SAMPLE_DATA=false; production start-up check passed
3	Same tag deployed to Production; technical smoke test passed; administrator password changed; two-step verification on
4	Configuration promoted with test personas removed and numbering set from the old system; on-screen items entered; comparison UAT against Production shows no change
5	Production backup taken; Pre-Prod restored from it at production topology, versions and non-secret settings (section 4.8); masked if people without production access take part
6	Scheduler and e-mail controlled for the rehearsal
7	Rehearsal done and timed; reconciliation signed; lock switched off again if a second rehearsal is needed

10.5 Checklist: go-live in Production

#	Check
1	GNG-1 passed; CAB approval of the cutover; release notes issued 5 business days ahead
2	Snapshot before the loads; goLive.cutover_date checked
3	Comparison Production against Pre-Prod shows no unapproved configuration change
4	If a smoke test ran in Production: transaction reset done and checked
5	Final migration loaded and reconciled; GNG-2 passed; goLive.locked on; schedules and e-mail on
6	Pre-Prod removal planned for hypercare exit and recorded in the environment sheet

10.6 Checklist: release after go-live

#	Check
1	Change request approved by the CAB, with test evidence and rollback
2	Release rehearsed on the same tag: major release in a Pre-Prod created from a Production backup (masked if needed) and removed afterwards; minor or patch release in UAT (and SIT for a large broker)
3	Snapshot taken in the maintenance window before deploying

#	Check
4	Deployed one instance first; <code>/api/version</code> shows the tag; no pending migration
5	Smoke test passed; manual actions of the release notes done
6	Previous image and front-end build kept for rollback

11. Decisions and sections to complete

#	Item	Decision of 04 October 2026	Where
1	Environment set by broker size	Small and medium: Dev, UAT, Production, with integration in Dev. Large: Dev, SIT, UAT, Production, with SIT and UAT separate	Section 4.2
2	Pre-Prod	Temporary for every size: created from a Production backup for the go-live rehearsal and each major release, then removed	Section 4.3
3	Hosting cost of SIT and Pre-Prod	Priced per environment in the pricing workbook and the Rate Card; Pre-Prod per month of use	Section 4.5
4	Front-end build	One build promoted through all environments, with runtime configuration	Section 5.5: to be completed with the release
5	Deployment jobs	Release pipeline with an approval gate per environment	Section 5.3: to be completed with the release
6	Comparison report between environments	Added	Section 6.6: to be completed with the release
7	Data masking	A masking tool is required	Section 4.9: to be completed with the release

12. Document Sign-Off

Serial Number	Description	Owner	Status
1	Sections marked "Section to be completed with the release" (front-end build, release pipeline, comparison report, data masking) are completed when those deliveries are released	iorta TechNXT DevOps lead	Open

Sign Off Date	Signature	Approved By	Department & Project Role



Insurance technology for the Philippines



10+

COUNTRIES

Global Presence



8

GLOBAL OFFICES

Strategically Located



11+

MARKETS SERVED

Across Continents



100%

COMMITMENT

To Excellence



MALAYSIA



INDIA



PHILIPPINES



THAILAND



VIETNAM



USA



ETHIOPIA



CAMBODIA



HONGKONG



SINGAPORE



Scan More Details